PART 1 – THEORETICAL QUESTIONS

Questions and answers

1. ReactJS and NextJS: Rendering and Strategies

1(a) Explain, in your own words, the differences between Client-Side Rendering (CSR), Server-Side Rendering (SSR), and Static Site Generation (SSG).

Client-Side Rendering (CSR) is when the browser is responsible for building most of the page. The server usually sends an initial HTML file, and JavaScript loads the data and renders the content. This approach is common in React applications. It can work well for applications where users interact with the system after logging in, such as dashboards. One downside is that the first load can take longer, especially if the page depends on a lot of JavaScript or API requests.

Server-Side Rendering (SSR) means that the server generates the page's HTML before sending it to the browser. The user can receive the main content earlier, and the browser then makes the page interactive by loading and running JavaScript. This can be useful for pages with frequently changing information, such as product pages in an online store. It can also help search engines understand the content. However, rendering pages on the server requires processing, so caching and server performance need to be considered.

Static Site Generation (SSG) means that the pages are generated in advance, usually when the project is built. The generated files can then be served quickly, for example through a CDN. This is a good option for content that does not change all the time, such as documentation, blog posts, or institutional pages. The main limitation is that when the content changes, the page may need to be generated again. Frameworks such as Next.js also offer revalidation, which makes it possible to update static content without rebuilding the entire application.

1(b) In which situations can each approach be useful or problematic? Give practical examples.

I would choose the rendering strategy based on the type of page and how often its information changes. For example, CSR is a reasonable choice for an internal dashboard or an administrative system, where the user is already logged in and search engine visibility is not a priority. It may be less suitable when the initial page content needs to appear quickly or be indexed by search engines.

SSR can be a better fit for pages where the content changes often and should be available as soon as the page opens. An online store is a practical example, because product information and prices can change, and the pages may also need to be indexed by search engines. The trade-off is that the server has more work to do, so performance and caching matter.

SSG is useful when the content changes less frequently. A company website, a blog, or a documentation site are good examples. Since the pages are already generated, they can be delivered quickly and with less work for the server. It becomes less convenient when the information changes constantly, unless a revalidation strategy is used.

These approaches are not always exclusive. In Next.js, for example, different pages can use different rendering strategies depending on their needs.

2. Backend: Concepts and Approaches

2(a) What are the main ways to implement the backend of a web application?

There are different ways to organize and implement a backend, and the choice depends on the size of the application, its requirements, and how the team will maintain it.

One common option is a **monolithic backend**, where the main features of the application are developed and deployed as one system. It is usually simpler to start with and can be a good choice for small or medium-sized applications. As the system grows, however, the code can become harder to maintain if its responsibilities are not well organized.

Another option is a **microservices architecture**. In this model, the application is divided into smaller services, each responsible for a specific part of the system. This can help teams develop and scale parts of the application independently. On the other hand, it adds complexity, because the services need to communicate and issues such as monitoring, deployment, and data consistency become more important.

A backend can also be built using **serverless functions**, where the cloud provider manages much of the infrastructure and runs functions when they are needed. This can be useful for specific tasks or applications with variable traffic. The developer still needs to consider execution limits, cold starts, and the way different functions communicate.

There are also different ways for the frontend and backend to communicate. **REST** is a widely used approach based on HTTP resources and methods. **GraphQL** allows the client to request the data it needs through a query. In some applications, a **Backend for Frontend (BFF)** is used to provide an API designed for a specific frontend, which can help avoid exposing the frontend to several backend services directly.

These options can be combined. For example, a system can have a monolithic backend that exposes a REST API, or use microservices behind a BFF. I would not choose an architecture only because it is more modern; I would consider the actual needs of the application and the complexity the team can handle.

2(b) Do you know tRPC? What are its advantages and disadvantages, especially regarding security, typing, and scalability?

I know the proposal behind **tRPC**: it is a tool for building APIs in TypeScript applications, allowing the frontend and backend to share types. Instead of defining a separate API contract and manually keeping the types aligned, the client can infer the input and output types from the backend procedures. This can reduce duplicated definitions and help catch mistakes during development.

One advantage is the developer experience, especially when both the frontend and backend use TypeScript. For example, if the input or return type of a procedure changes, the client can show the type error during development. This makes it easier to keep the two sides consistent.

However, TypeScript types do not replace runtime validation. Types are removed when the code is executed, so data received from a request still needs to be validated. A library such as Zod can be used for this. Security also depends on implementing authentication and authorization correctly. Using tRPC does not automatically protect an endpoint or guarantee that a user can access only the data they are allowed to see.

Another point to consider is that tRPC is especially convenient when the client and server are part of the same TypeScript ecosystem. It may be less suitable when an API needs to be consumed by different languages or by external clients, where a more standardized contract such as REST with OpenAPI may be a better fit.

Regarding scalability, I would not say that tRPC itself makes an application more or less scalable. That depends much more on the backend architecture, infrastructure, database, and how requests are handled. tRPC can be used in a scalable application, but the team still needs to design those parts carefully.

So, I see tRPC as a useful option for TypeScript projects that benefit from sharing types between the frontend and backend. The main decision is whether that convenience fits the project and the systems that need to consume its API.

Note: The answers are written to explain the concepts clearly without suggesting hands-on experience with tRPC beyond the level described.